

PARACASYA_RESUMEN_TECNICO_REAL

Documento técnico para migración del proyecto

1. Estado general del proyecto

Proyecto: ParacasYa Market Online

Carpeta local: C:\Users\Usuario\Documents\ParacasYa Market Online

Rama actual: main

Último commit: 90ea544 feat: agregar catalogo de cervezas

Estado Git: limpio

Archivos modificados: ninguno

Archivos sin trackear: ninguno

Working tree: limpio

Durante la auditoría técnica no se modificaron archivos, no se hizo commit, no se hizo push y no se tocaron secretos.

2. Stack técnico

ParacasYa Market Online está construido con:

- **Frontend:** React 19.2.6
- **Build tool:** Vite 8.0.14
- **Router:** react-router-dom 7.15.1
- **Base de datos:** Supabase
- **Cliente Supabase:** @supabase/supabase-js 2.106.2
- **Iconos/UI:** lucide-react
- **QR:** qrcode
- **CSS:** Tailwind CSS 4.3.0 vía @tailwindcss/vite

Scripts disponibles

```
npm run dev
npm run build
npm run preview
```

3. Estructura principal de carpetas

Carpeta	Función
<code>src/</code>	Código principal de la app React
<code>src/pages/</code>	Páginas principales: landing, cliente, caja y admin
<code>src/components/</code>	Componentes reutilizables: carrito, tarjetas, header, recibo, QR y protección PIN
<code>src/services/</code>	Servicios de Supabase, pedidos, catálogo, admin y tienda abierta/cerrada
<code>src/utils/</code>	Helpers de WhatsApp, QR, origen, moneda, totales y acceso interno
<code>src/data/</code>	Fallback mock local para productos y categorías
<code>src/styles/</code>	CSS global
<code>supabase/</code>	SQL local, notas de seguridad y Edge Functions
<code>supabase/functions/</code>	Edge Functions <code>admin-catalog</code> y <code>notify-order-telegram</code>

Actualmente no existe carpeta `public/`.

4. Rutas principales de la aplicación

Ruta	Archivo	Función
<code>/</code>	<code>src/pages/LandingPage.jsx</code>	Landing pública principal
<code>/cliente</code>	<code>src/pages/ClientePage.jsx</code>	Tienda pública, carrito, checkout y recibo premium
<code>/caja</code>	<code>src/pages/CajaPage.jsx</code>	Panel interno de pedidos, protegido por PIN
<code>/admin</code>	<code>src/pages/AdminPage.jsx</code>	Panel interno de catálogo, tienda y QR, protegido por PIN
<code>*</code>	<code>src/App.jsx</code>	Redirige a <code>/</code>

No existe una ruta separada para seguimiento de pedido.
El seguimiento vive dentro del componente:

```
src/components/OrderSuccessReceipt.jsx
```

El origen por hotel o QR usa este formato:

```
/cliente?origen=hotel-paracas
```

5. Supabase

El cliente de Supabase se crea en:

```
src/lib/supabaseClient.js
```

Variables frontend usadas

Variable	Uso
VITE_SUPABASE_URL	URL base del proyecto Supabase
VITE_SUPABASE_ANON_KEY	Anon key para lectura pública y pedidos
VITE_INTERNAL_NOTIFY_TOKEN	Token frontend para invocar notificación Telegram
VITE_INTERNAL_PIN	PIN interno opcional para <code>/caja</code> y <code>/admin</code>

El cliente normaliza la URL con:

```
new URL(value).origin
```

Esto evita errores cuando la URL viene con `/rest/v1`.

Tablas detectadas

- products
- categories
- orders
- order_items
- store_settings

Edge Functions detectadas

- admin-catalog
- notify-order-telegram

SQL local relevante

- supabase/paracasya_products_categories.sql
- supabase/paracasya_store_settings.sql

- `supabase/paracasya_order_origin.sql`
 - `supabase/paracasya_beer_catalog.sql`
 - `supabase/admin_catalog_security_notes.md`
-

6. Tablas principales

`categories`

Tabla usada para las categorías del catálogo.

Columnas inferidas

- `id`
- `name`
- `slug`
- `icon`
- `is_active`
- `sort_order`
- `created_at`
- `updated_at`

Uso

- `/cliente` la usa para filtros visibles.
- `/admin` la usa para crear y editar categorías.
- `products` la usa como relación.

RLS local: SELECT público solo para categorías activas.

`products`

Tabla usada para el catálogo real de productos.

Columnas inferidas

- `id`
- `name`
- `description`
- `price`
- `category_id`
- `category_slug`
- `image_url`
- `is_available`
- `is_featured`

- `stock_status`
- `created_at`
- `updated_at`

Uso

- `/cliente`
- `/admin`
- fallback visual con `ProductCard`

RLS local: SELECT público solo para productos disponibles.

Las escrituras de catálogo se hacen mediante Edge Function `admin-catalog`.

orders

Tabla usada para pedidos reales.

Columnas inferidas

- `id`
- `created_at`
- `order_code`
- `customer_name`
- `customer_phone`
- `location_name`
- `room_reference`
- `payment_method`
- `delivery_notes`
- `status`
- `subtotal`
- `delivery_fee`
- `total`
- `order_origin`
- `order_origin_label`

Uso

- Creación desde `/cliente`.
- Gestión en `/caja`.
- Seguimiento visual en el recibo.
- Alertas Telegram.

order_items

Tabla usada para guardar los productos de cada pedido.

Columnas inferidas

- order_id
- product_id
- product_name
- quantity
- unit_price
- total_price

Uso

- Creación de pedidos.
- Lectura de pedidos en /caja .
- Alerta Telegram.
- Recibo premium.

store_settings

Tabla usada para tienda abierta/cerrada.

Columnas inferidas

- id
- store_open
- opening_hours
- closed_message
- created_at
- updated_at

Uso

- /cliente bloquea pedidos si la tienda está cerrada.
- /admin cambia el estado de tienda.
- Servicio relacionado: storeSettingsService.js .

7. Productos y catálogo

Los productos se cargan desde:

```
src/services/productsService.js
```

Flujo de productos en `/cliente`

1. `/cliente` intenta ejecutar `getAvailableProducts()`.
2. Consulta `products` con `is_available = true`.
3. Ordena destacados primero.
4. Si Supabase falla o está vacío, usa fallback local:
5. `src/data/products.js`
6. `src/data/categories.js`

Categorías

Las categorías se cargan desde:

```
src/services/categoriesService.js
```

`/cliente` usa `getActiveCategories()` y filtra:

```
categories.is_active = true
```

`/admin` puede listar todas mediante Edge Function si hay token.

Disponibilidad

- `is_available = true`: aparece en tienda.
- `is_available = false`: queda oculto en `/cliente`, pero visible en `/admin`.

Destacados

- `is_featured = true`: se muestran como recomendados o populares.

Imágenes

Las imágenes se guardan en:

```
products.image_url
```

El frontend normaliza ese campo como:

```
imageUrl
```

`ProductCard.jsx` muestra imagen si existe.
Si la imagen falla, vuelve a un placeholder visual.

8. Catálogo de cervezas

Existe el archivo:

```
supabase/paracasya_beer_catalog.sql
```

Este archivo contiene una inserción segura e idempotente de:

- Categoría: **Bebidas alcohólicas**
- Slug: `bebidas-alcoholicas`
- 8 productos de cerveza
- `is_available = true`
- `stock_status = available`
- `image_url = null`
- Aviso de venta responsable para mayores de 18 años

Productos agregados:

1. Cerveza Pilsen lata 355 ml — S/7.00
 2. Cerveza Cusqueña lata/botella — S/8.50
 3. Six pack Cusqueña — S/42.00
 4. Six pack Pilsen lata 355 ml — S/35.00
 5. Corona botella 355 ml — S/11.00
 6. Six pack Corona botella 355 ml — S/60.00
 7. Tres Cruces lata/botella — S/7.50
 8. Stella Artois botella/lata — S/10.00
-

9. Cloudinary

Estado actual:

Cloudinary no está integrado como flujo de subida automática.

No se detectaron:

- Variables Cloudinary.
- Presets unsigned.

- Servicio de upload.
- Edge Function de imágenes.
- Subida directa desde `/admin`.

Lo que sí existe:

- Campo manual en `/admin` para pegar URL de imagen.
- Esa URL se guarda como `image_url`.
- Puede ser una URL de Cloudinary, pero el proyecto no sube imágenes automáticamente.

Recomendación futura

Si se integra Cloudinary después:

- Hacerlo con upload seguro.
- Validar tipo y tamaño de archivo.
- No poner secretos privados en frontend.
- Considerar una Edge Function o preset controlado.

10. Pedidos

Archivo principal:

```
src/services/ordersService.js
```

Flujo cliente

1. El cliente agrega productos al carrito.
2. Completa:
3. Nombre.
4. WhatsApp.
5. Hotel/Airbnb/dirección.
6. Habitación o referencia.
7. Comentarios.
8. Método de pago.
9. `createOrder()` calcula totales.
10. Inserta primero en `orders`.
11. Luego inserta productos en `order_items`.
12. Genera código tipo:

```
PYA-${Date.now()}-${random}
```

1. Guarda recibo local en `localStorage`.
2. Muestra recibo premium.
3. Dispara alerta Telegram en segundo plano.

Campos enviados a `orders`

- `order_code`
- `customer_name`
- `customer_phone`
- `location_name`
- `room_reference`
- `payment_method`
- `delivery_notes`
- `status`
- `subtotal`
- `delivery_fee`
- `total`
- `order_origin`
- `order_origin_label`

11. Caja

`getOrders()` lee:

```
orders con order_items(*)
```

Ordena por:

```
created_at descendente
```

`updateOrderStatus()` cambia el estado del pedido.

Estados válidos

- Pendiente
- Confirmado
- Preparando
- En camino
- Entregado
- Cancelado

Funciones actuales de `/caja`

- Lista pedidos reales desde Supabase.
- Muestra métricas:
 - pendientes,
 - entregados hoy,
 - vendido hoy.
- Permite cambiar estados.
- Muestra origen QR si existe.
- Muestra comentarios del cliente.
- Abre WhatsApp para confirmar pedido.

Limitaciones actuales de caja

- No hay Supabase Realtime.
 - No hay polling automático.
 - No se detectó sonido de nuevo pedido.
 - Se actualiza manualmente con botón “Actualizar”.
-

12. Telegram

Telegram se dispara después de guardar `orders` y `order_items`.

Edge Function

```
notify-order-telegram
```

Header usado

```
x-internal-notify-token
```

Variable frontend

```
VITE_INTERNAL_NOTIFY_TOKEN
```

Si falla Telegram, el pedido no falla.

Variables esperadas en Supabase Function

- `INTERNAL_NOTIFY_TOKEN`
- `TELEGRAM_BOT_TOKEN`
- `TELEGRAM_CHAT_ID`

Seguridad

La función valida el header `x-internal-notify-token`, escapa HTML y no debe imprimir secretos.

Riesgo detectado:

`VITE_INTERNAL_NOTIFY_TOKEN` vive en frontend y por definición es visible en el bundle. Es suficiente como barrera MVP, pero no como seguridad fuerte.

13. WhatsApp

Archivo principal:

```
src/utils/whatsapp.js
```

Normalización de número

- Si tiene 9 dígitos y empieza con 9, agrega prefijo `51`.
- Si ya viene como `519XXXXXXX`, lo conserva.
- Si no cumple, devuelve `null`.

Link usado

```
https://wa.me/{numero}?text={mensaje}
```

Uso

- `OrderCard.jsx` en `/caja`.
- Botón: "Confirmar por WhatsApp".
- Si el número no es válido, muestra "WhatsApp no válido".

No se detectaron mensajes automáticos por estado.

El mensaje actual es de confirmación general del pedido.

14. QR / origen hotel

Archivos relacionados:

- `src/utils/orderOrigin.js`
- `src/utils/partnerQrData.js`
- `src/components/PartnerQrCard.jsx`
- `src/components/PartnerQrCreator.jsx`

Funcionamiento

1. Detecta `?origen=...` en `/cliente`.
2. Normaliza el slug.
3. Guarda origen en `localStorage` con key:

```
paracasya_order_origin
```

1. Muestra:

```
Pedido asociado a: Hotel Paracas
```

1. Envía al pedido:
2. `order_origin`
3. `order_origin_label`
4. `/caja` muestra el origen en la tarjeta del pedido.

Generador QR

Vive en `/admin`.

Usa dependencia:

```
qrcode
```

Crea enlaces tipo:

```
/cliente?origen=hotel-paracas
```

Permite:

- Copiar link.
- Copiar mensaje.
- Imprimir.
- Descargar PNG.

Los materiales personalizados se guardan en `localStorage`.

No hay tabla específica de QR.

El origen se registra en `orders`.

15. Seguimiento de pedido

Componente:

```
src/components/OrderSuccessReceipt.jsx
```

No existe página separada.

Identificación

Usa:

- `orderCode`
- opcionalmente `id`

Servicio

```
getOrderStatus({ orderCode, orderId })
```

Polling

- Cada 8 segundos.
- Se detiene al desmontar.
- También se detiene en estados terminales:
- entregado,
- cancelado.

Estados visuales

- Pedido recibido.
- Confirmación por WhatsApp.
- Preparación.
- Entrega.

Mapeo aproximado

Estado	Paso visual
Pendiente	Paso 1
Confirmado	Paso 2
Preparando	Paso 3
En camino	Paso 4

Estado	Paso visual
Entregado	Paso 4
Cancelado	Estado cancelado

También funciona con recibo recuperado desde `localStorage`.

16. Admin

Archivo principal:

```
src/pages/AdminPage.jsx
```

Protección

`/admin` está protegido por PIN frontend.

Archivos:

- `src/components/ProtectedInternalRoute.jsx`
- `src/utils/internalAccess.js`

PIN:

```
VITE_INTERNAL_PIN
```

Fallback:

```
2468
```

Importante:

Esto es protección MVP, no seguridad real.

Funciones de `/admin`

Permite:

- Crear productos.
- Editar productos.
- Cambiar precio.

- Pegar `image_url`.
- Activar/ocultar productos.
- Marcar destacados.
- Crear/editar categorías.
- Activar/desactivar categorías.
- Configurar tienda abierta/cerrada.
- Generar material QR.

Escrituras

No usa anon directo para catálogo.

Usa Edge Function:

```
admin-catalog
```

Header:

```
x-admin-token
```

Token guardado en sessionStorage:

```
paracasya_admin_catalog_token
```

Tienda abierta/cerrada

Usa `store_settings`.

Guarda vía `admin-catalog`.

Tiene fallback local si falla persistencia.

17. Edge Functions

```
admin-catalog
```

Archivo:

```
supabase/functions/admin-catalog/index.ts
```

Propósito

Escritura segura de catálogo/admin.

Usa `service_role` solo dentro de Supabase Functions.

Variables esperadas

- ADMIN_CATALOG_TOKEN
- SUPABASE_URL o PROJECT_URL
- SUPABASE_SERVICE_ROLE_KEY o SERVICE_ROLE_KEY

Acciones

- listProducts
- createProduct
- updateProduct
- toggleProductAvailability
- toggleProductFeatured
- listCategories
- createCategory
- updateCategory
- toggleCategoryActive
- getStoreSettings
- updateStoreSettings

Riesgo

ADMIN_CATALOG_TOKEN es el control actual.
Mejora futura: Supabase Auth con roles reales.

notify-order-telegram

Archivo:

```
supabase/functions/notify-order-telegram/index.ts
```

Propósito

Enviar alerta interna a Telegram cuando entra un pedido.

Variables esperadas

- INTERNAL_NOTIFY_TOKEN
- TELEGRAM_BOT_TOKEN
- TELEGRAM_CHAT_ID

Caller

```
src/services/ordersService.js
```

Seguridad

- Valida header `x-internal-notify-token`.
- Escapa HTML.
- No debe imprimir secretos.
- Si falla, el pedido igual se crea.

18. Variables de entorno

Variable	Dónde se usa	Para qué sirve	Sensibilidad	Local	Vercel	Supabase Function
<code>VITE_SUPABASE_URL</code>	Frontend	Conectar Supabase	Media	Sí	Sí	No
<code>VITE_SUPABASE_ANON_KEY</code>	Frontend	Anon key pública	Media	Sí	Sí	No
<code>VITE_INTERNAL_NOTIFY_TOKEN</code>	Frontend	Invocar Telegram Function	Alta aunque sea VITE	Opcional/ideal	Sí	No
<code>VITE_INTERNAL_PIN</code>	Frontend	PIN MVP interno	Baja/media	Opcional	Opcional	No
<code>ADMIN_CATALOG_TOKEN</code>	Edge Function	Autorizar admin-catalog	Alta	No	No	Sí
<code>SUPABASE_URL</code>	Edge Function	URL Supabase	Media	No	No	Opcional
<code>PROJECT_URL</code>	Edge Function	Fallback URL Supabase	Media	No	No	Sí
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Edge Function	Service role	Crítica	No	No	Opcional
<code>SERVICE_ROLE_KEY</code>	Edge Function	Fallback service role	Crítica	No	No	Sí

Variable	Dónde se usa	Para qué sirve	Sensibilidad	Local	Vercel	Supabase Function
<code>INTERNAL_NOTIFY_TOKEN</code>	Edge Function	Autorizar Telegram	Alta	No	No	Sí
<code>TELEGRAM_BOT_TOKEN</code>	Edge Function	Bot Telegram	Crítica	No	No	Sí
<code>TELEGRAM_CHAT_ID</code>	Edge Function	Chat destino	Alta	No	No	Sí

No se detectaron variables Cloudinary.

19. Seguridad

Puntos correctos

- No hay `service_role` en `src/`.
- Catálogo admin escribe por Edge Function.
- SQL local habilita RLS en `products`, `categories`, `store_settings`.
- SELECT público limitado para catálogo visible.
- No se detectaron políticas locales de escritura anon para catálogo.
- `paracasya_order_origin.sql` no cambia RLS.
- `paracasya_beer_catalog.sql` no cambia RLS ni abre escrituras.
- No se detectó DELETE implementado en admin.

Puntos sensibles

- PIN frontend no es seguridad fuerte.
- `VITE_INTERNAL_NOTIFY_TOKEN` es visible en frontend.
- `ADMIN_CATALOG_TOKEN` en sessionStorage depende de operación responsable.
- Pedidos se crean con anon key, aceptable en MVP si las políticas de `orders` y `order_items` están controladas.
- Telegram y service role deben vivir solo en Supabase Functions.

No tocar sin cuidado

- RLS.
- Edge Functions.
- Secrets.
- `.env`.
- `.env.local`.
- Service role.
- Políticas de escritura.
- Tabla `orders`.

20. Estado de Fase 3D

Estado actual del repo: limpio.

No hay cambios pendientes de Fase 3D.

Evidencias actuales en `/cliente`

Elemento buscado	Estado
Bloque superior comercial	Existe hero comercial actual
Tienda abierta/cerrada visible	Sí, badge y bloque de tienda cerrada
Sección "Más pedidos de noche"	No con ese nombre exacto
Sección recomendados	Sí: "Más pedidos para tu estadía"
Categorías con iconos	Parcial: <code>CategoryPill</code> usa presentación/short/tone
Aviso alcohol +18 visible	No como bloque general; sí en descripciones de cervezas
Mensaje de confianza en checkout	Parcial
Mejoras mobile carrito	Sí, botón flotante mobile cuando hay items

Riesgo:

Si se reintenta Fase 3D, conviene trabajar con una referencia más cercana al estilo actual, porque las mejoras visuales anteriores fueron revertidas por preferencia estética.

21. Build y validación

Comando ejecutado:

```
npm run build
```

Resultado:

```
✓ built in 1.17s
```

Warning:

```
Some chunks are larger than 500 kB after minification.
```

No es error.

Recomendación futura:

Aplicar code splitting o dynamic imports si el bundle crece más.

22. Git

Últimos 5 commits

```
90ea544 feat: agregar catalogo de cervezas
8c073e2 feat: agregar alerta interna por Telegram
0ab436e feat: mejorar seguimiento visual de pedidos
0ff349e feat: mejorar experiencia de confirmacion de pedidos
35a618d feat: agregar recibo digital de pedido
```

Estado actual

```
git status --short
```

Sin salida.

No hay archivos sensibles modificados.

No hay cambios para commitear.

23. Riesgos actuales

- Muchos productos pueden no tener imágenes reales.
 - Cloudinary todavía no está integrado como upload.
 - Login real no existe; PIN frontend es MVP.
 - Token admin depende de operación manual.
 - Telegram usa token VITE en frontend para invocar Function; no es seguridad fuerte.
 - Caja no tiene Realtime ni sonido de nuevo pedido.
 - Delivery fee parece fijo/calculado localmente, no zonificado.
 - No hay reportes de ventas avanzados.
 - No hay panel de estadísticas por hotel/origen QR.
 - Venta de alcohol necesita controles operativos reales para mayores de 18 años.
 - Catálogo depende de mantenimiento manual desde admin.
 - No hay roles de usuario ni auditoría por operador.
-

24. Recomendaciones técnicas inmediatas

1. Probar pedido real completo en producción

Validar:

- `/cliente`
- `/caja`
- Telegram
- WhatsApp
- Seguimiento del recibo

2. Subir imágenes reales

Primero:

- Pegando URLs Cloudinary manuales.

Después:

- Implementar upload formal.

3. Agregar bloque visible de alcohol responsable

Especialmente si `bebidas-alcoholicas` está activa.

Texto recomendado:

Venta solo para mayores de 18 años. Podemos solicitar confirmación de edad al entregar.

4. Mejorar caja

Agregar en futuras fases:

- Polling automático o Supabase Realtime.
- Alerta sonora opcional.
- Filtro por estado.

5. Agregar reportes simples

Medir:

- Ventas por día.
- Productos más vendidos.
- Pedidos por origen QR.

6. Mejorar seguridad

Más adelante:

- Supabase Auth.
- Roles admin/caja.
- Reemplazar PIN frontend.
- Mover notificación Telegram a un flujo más server-side si escala.

7. Mejorar delivery

Agregar:

- Zonas.
- Tarifa configurable por zona.
- Horarios por día.

8. Revisar bundle

Aplicar code splitting futuro para reducir warning de chunk grande.

25. Resumen ejecutivo final

ParacasYa Market Online está funcionando como un MVP avanzado.

Actualmente cuenta con:

- Landing pública.
- Tienda `/cliente` con catálogo Supabase y fallback local.
- Carrito.
- Checkout.
- Comentarios.
- Recibo premium.
- Último pedido recuperable desde el mismo navegador.
- Seguimiento visual conectado al estado real de `/caja`.
- Pedidos reales en Supabase.
- Caja operativa con estados y WhatsApp.
- Admin real de catálogo vía Edge Function segura.
- Tienda abierta/cerrada.
- QR por hotel/Airbnb.
- Telegram interno al entrar pedidos.
- Catálogo de cervezas aplicado y versionado.

Pendientes principales

- Login real.

- Realtime o alertas en caja.
- Cloudinary upload real.
- Reportes operativos.
- Mayor control para alcohol.
- Mejoras visuales futuras con dirección estética validada antes de implementar.

No se debe tocar sin confirmar

- `.env`
- `.env.local`
- Secrets
- RLS
- Edge Functions
- Supabase remoto
- Service role
- Políticas de escritura
- Tablas de pedidos

Próxima fase recomendada

Realizar una prueba operativa real en producción con un pedido desde hotel/origen QR, validando:

1. Pedido desde `/cliente`.
2. Registro en Supabase.
3. Aparición en `/caja`.
4. Alerta Telegram.
5. Botón WhatsApp.
6. Seguimiento del recibo de punta a punta.